Stack Smashing Defense: A Buffer Overflow Lab Exercise

William E. Byrd
webyrd@indiana.edu

Computer Science Department
University of Maryland, Baltimore County

February 14, 2004

[Working Draft]

Abstract. The new Cyber Defense Lab (CDL) at UMBC provides an isolated environment where students can engage in administrator-level and insecure activities. This paper presents a lab exercise for a 300-level undergraduate course that will give students a better understanding of buffer overflow attacks, and how to write software that is resistant to such attacks. The exercises are designed to be used with Red Hat Linux 9 and IBM's Stack Smashing Protector modification to the GCC compiler suite. The materials provided include a summary sheet, setup information for instructors, grading suggestions, and suggestions and resources for an accompanying lecture.

Keywords. Security education, lab exercise, buffer overflow, stack smashing, cyber defense lab, Stack Smashing Protector.

1 Introduction

The new Cyber Defense Lab (CDL) at UMBC provides a unique opportunity for students to work with topics and technologies that would ordinarily be considered too dangerous to investigate in other computer labs. In order for students to benefit from this lab, existing courses need to include material designed to use the lab's resources. In order to lighten the workload of instructors, the CDL has developed several lab exercises usable by instructors of a variety of UMBC courses.

According to recent statistics from Carnegie Mellon's Computer Emergency Response Team, buffer overflow vulnerabilities account for about 50% of all reported security vulnerabilities [2]. Most buffer overflow attacks compromise the victim application by overwriting the return address of the currently executing function with an address that points to malicious code stored at the beginning of the buffer. Typically the malicious code spawns a command shell with `root` level access. Other types of buffer overflow attacks are possible, such as application-specific attacks that overwrite local variables located higher up on the stack, corrupting the data used by the victim application.

2 Background and Previous Work

Several different methods exist for preventing and detecting buffer overflow attacks, including array bounds checking [9], annotation-assisted static analysis [13, 7], non-executable runtime stacks [8, 5, 12], and dynamically loadable libraries that implement secure versions of standard library routines [2]. One of the most important mechanisms is the canary-based stack protection method introduced by Cowan, et al., [3] and improved upon by Etoh [4]. Canary-based stack protection mechanisms work by pushing a known value, or canary, onto the runtime stack whenever a function call is made by the running application. This canary value is located below the return address on the stack. Before the function returns, the program checks if the canary value has been modified. If the canary has been modified, a buffer overflow may have occurred, and the program is terminated with an error message. If the canary value is intact, the function returns normally, and the program continues execution.

Existing canary-based stack protection mechanisms check the canary value only immediately before the currently executing function returns. Unfortunately, an attacker can bypass this protection by overflowing a stack-based buffer and overwriting local variables located higher up on the stack. Although this buffer overflow may be detected at return time, the executing function is free to perform operations on the corrupted data on the stack, resulting in a potential security vulnerability. For example, imagine an accounting system in which a variable on the stack contains the value of an employee's salary. An attacker could overwrite this variable, changing the employee's salary. The currently executing function could electronically transfer this amount to the employee's bank account in the Cayman Islands before the function returns and the buffer overflow is detected.

2.1 Anatomy of a Buffer Overflow Attack

Buffer overflows are common in programs written in languages such as C and C++ that do not perform array bounds checking. Figure 1 shows a typical stack frame after a C function `foo()` has been called by another function, `bar()`. Note that the return address for `foo()` is saved on the stack. After `foo()` has finished executing, flow of control passes to the code located at this return address (presumably within the calling function, `bar()`).

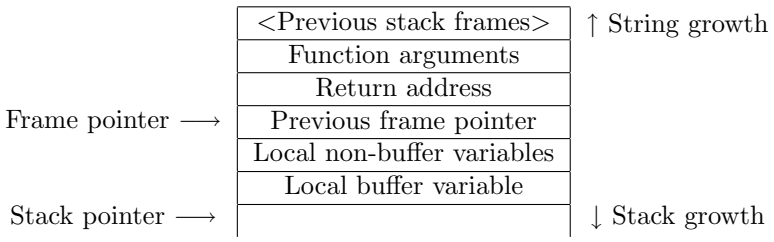


Figure 1: Typical stack frame immediately before function execution. The function's arguments have been pushed onto the stack, followed by the address of the calling function and a pointer to the previous stack frame. Space for local variables is then allocated on the stack. In this case, several non-buffer local variables are located above a local buffer variable.
Note that an attacker who overflows the buffer can overwrite the function's return address, altering the flow of control of the victim application.

The most common type of buffer overflow attack is known as *stack smashing* [1]. In a stack smashing attack, the attacker overflows a buffer with a string containing malicious code. The end of the string contains the address of the buffer. If the attacker chooses the proper string length, the buffer address will overwrite the function's return address on the stack (see Figure 2). When the function returns, flow of control will be passed to the malicious code instead of being passed back to the function's caller. Typically, the malicious code will spawn a command-line shell with the same security privileges as the exploited application.

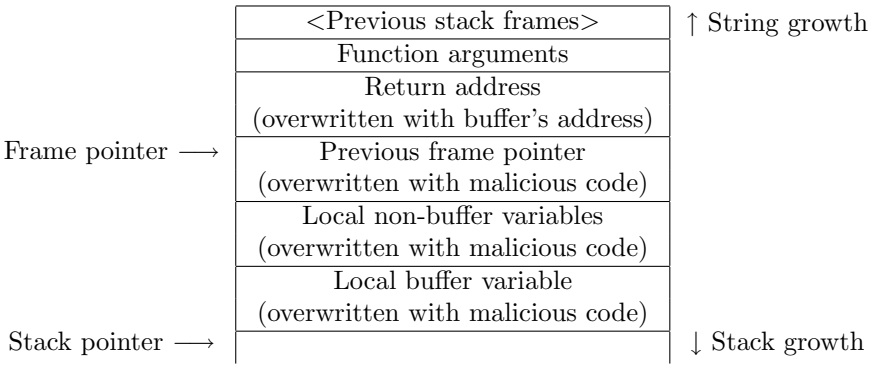


Figure 2: Stack frame during a stack smashing attack, immediately after the buffer has been overflown by a string containing malicious code followed by the absolute address of the buffer. When the function returns, flow of control will be transferred to the malicious code held in the buffer.
Note that the attacker must carefully choose the length of the string to ensure that the function's return address will be overwritten with the address of the buffer.

2.2 Existing Stack Protection Mechanisms

Several mechanisms for preventing buffer overflow attacks have been proposed [3][2]. For the purposes of this paper, all of these mechanisms can be broken into two separate categories: those that place canary values on the stack and those that do not.

Canary-based mechanisms do not prevent all buffer overflow attacks, but generally offer excellent runtime performance and represent the most popular approach to stack protection.

Of the non-canary-based approaches, three offer complete protection against buffer overflows, but require either human intervention (annotation-assisted analysis) or impose significant performance penalties at runtime (safe languages and bounds-checking compilers). Safe libraries and non-executable stacks represent efficient non-canary-based approaches, but do not protect against all buffer overflow attacks.

2.2.1 Canary-based Stack Protection Mechanisms

Canary-based stack protection mechanisms work by inserting a known value, or *canary*, onto the runtime stack below the return address of the executing function. When the function finishes execution, the stack protection mechanisms checks the canary value. If the canary value has been modified, a buffer overflow attack may be in progress, and the application is terminated with an error message. If the canary value is intact, the function returns normally, and the program continues execution.

Three popular canary-based products include StackGuard, SSP, and libverify.

StackGuard Cowan, et al., [3] were the first to present the idea of using canary values to protect against stack smashing attacks; their *StackGuard* implementation is an extension to the GNU C compiler (`gcc`) that instruments `gcc`-generated code with canary-based buffer overflow checks. Note that in order to protect a program with StackGuard the program must be recompiled with the StackGuard compiler, which in turn required access to the program's source code. WireX Communications, Inc., produces Immunix Secured Linux, an entire GNU/Linux distribution in which all programs have been compiled using the StackGuard compiler [14].

StackGuard protects the return address by pushing a randomly chosen canary value onto the stack immediately below the return address on function entry. Before the function returns to the caller, the validity of this canary is checked; the program is terminated if a change to the canary value is detected. The StackGuard protection model is shown in Figure 3.

Although StackGuard protects the return address, the saved frame pointer and local variables remain unprotected. Richarte [10] has shown how a clever attacker can use these limitations to defeat the StackGuard protection mechanism.

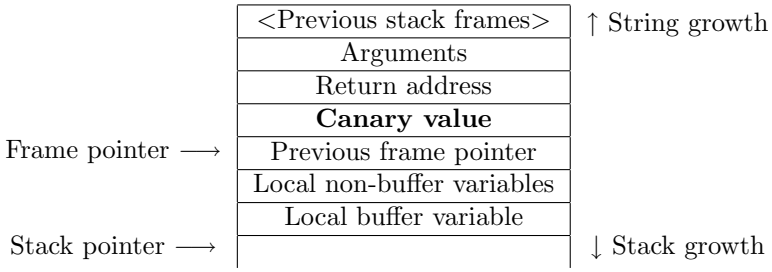


Figure 3: Typical stack frame from a program protected by the StackGuard compiler. A known value, or *canary*, is pushed onto the stack below the function's return address. The value of the canary is checked immediately before the function returns; if the value has been changed, the program exits with an error message.
Any buffer overflow attack that modifies the canary value will be detected at function return time. This protection model prevents stack smashing attacks that alter a program's flow of control by overwriting the function's return address. Unfortunately, poor placement of the canary allows an attacker to overwrite the saved frame pointer without fear of detection, permitting more subtle stack smashing attacks.

Stack Smashing Protector (SSP) Etoh's [4] *Stack Smashing Protector* (SSP) improves upon the StackGuard protection mechanism by moving the canary below the frame pointer, preventing the frame pointer stack smashing attack identified by Richarte [10]. Etoh's mechanism also re-orders data on the stack—all non-buffer local variables within a stack frame are placed below all buffer variables within that stack frame. This re-ordering prevents a buffer overflow from overwriting non-buffer variables local to the currently executing function. This mechanism does not, however, prevent an attacker from overwriting local variables in calling functions, which are located higher up on the stack. And, once again, the canary value is checked only immediately before returning from a function call. A typical stack frame for an SSP-protected program is shown in Figure 4.

As with the StackGuard compiler, a program must be recompiled with SSP in order to insert the stack protection code.

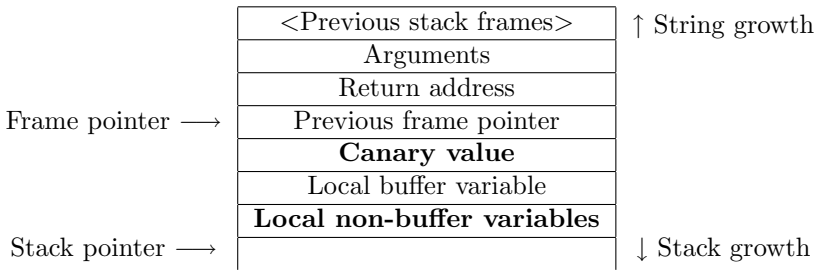


Figure 4: Typical stack frame from a program protected by Etoh's Stack Smashing Protector (SSP) compiler. In contrast with the StackGuard compiler, SSP places the canary below the saved frame pointer, preventing a known type of stack smashing attack. Also note that SSP rearranges local variables on the stack, ensuring that all buffer variables are located above all non-buffer variables within the stack frame. Rearranging the buffer variables on the stack prevents an attacker from overwriting local non-buffer variables.
Note that an attacker can still overwrite non-buffer variables in stack frames located higher up on the stack. Also note that SSP-protected programs check the canary value only at function return time, allowing an attacker to launch application-specific attacks that cause damage before the function returns.

Libverify *Libverify* [2] is a dynamically loadable library that offers canary-based stack protection similar to that provided by the StackGuard compiler. The primary advantage of libverify is that a program can be protected from buffer overflow attacks without recompiling the program's source code.

Libverify uses a protection scheme similar to that of StackGuard, but uses a different implementation. StackGuard introduces canary verification code at compile time, whereas libverify injects the verification code at the start of the process execution via binary re-writing of the process memory. Furthermore, libverify uses the actual return address as the canary value, instead of using a randomly chosen value.

Libverify suffers from two of the same security weaknesses as StackGuard as SSP: the canary value is checked only at function return time, and it is possible for an attacker to overwrite local variables located higher on the stack than the overflowed buffer.

2.2.2 Non-canary-based Stack Protection Mechanisms

Not all buffer overflow protection mechanisms rely on checking a canary value at function return time. The following stack protection methods offer different advantages and disadvantages than canary-based approaches.

Safe Languages Programs written in C and C++ are vulnerable to buffer overflow attacks because these languages do not perform automatic array bounds checking. Higher level languages, such as Lisp and Java, perform additional checks at runtime to ensure that all array operations properly respect array boundaries.

Writing a program in a "safe" language like Lisp or Java completely eliminates the possibility of buffer overflow attacks, at the cost of the performance overhead required to implement bounds checking; this performance cost can be significant [9]. Another disadvantage of using safe languages is that a great deal deal of software have already been written in C and C++—rewriting this software in a higher level programming language is infeasible.

Bounds-checking Compilers Jones and Kelly have developed a `gcc` patch that does full array bounds checking for C programs, preventing all possible buffer overflow attacks [3]. Unfortunately, as is the case with "safe" languages, array bounds checking often incurs a significant performance penalty. Protecting an existing C program also requires recompilation of the program, and requires that the program's source code is available.

Lhee and Chapin [13] have also developed an array bounds checking C compiler. Although Lhee and Chapin claim their implementation is efficient, they currently use their compiler only to protect certain unsafe string functions in the standard C library.

Ramarao, et. al., [9] have proposed a new "fast" array bounds checking mechanism for C programs in which only certain pointer dereferences need to be checked, although they have not provided performance data to support their claims.

Annotation-assisted Static Analysis Larochelle and Evans [7] have developed an extended version of the LCLint annotation-assisted static source code analysis tool that can detect buffer overflow vulnerabilities at compile time. Unfortunately, statically detecting buffer overflow vulnerabilities is an undecidable problem [7], so the method presented by Larochelle and Evans requires that a human provide additional source code annotation. This limitation also makes this approach much less attractive for protecting existing "legacy" applications from buffer overflow attacks.

Non-executable Stack Another method for preventing stack smashing attacks is to mark the stack area of memory as being non-executable. Recall that a stack smashing attack will only be effective if the attacker can both inject malicious code into a buffer and alter the flow of control of the program in order to execute this code. Making the stack non-executable prevents any malicious code injected onto the stack from being executed, preventing the most common type of stack smashing attack. It is also possible to mark the `malloc()`'d heap area as non-executable, to prevent more subtle types of stack smashing attacks.

Marking the stack as non-executable has its disadvantages. In order to implement a non-executable stack, it is usually necessary to patch the operating system kernel, which is not an option for "closed source" operating systems such as Windows. In addition, programs written in dynamic languages such as Lisp and Objective C need to store executable code on the stack; making the stack non-executable prevents these programs from running properly [2]. Finally, there are ways to bypass non-executable stacks and heaps by changing the flow of control of a program to point to a library routine [11].

Non-executable stack implementations for Linux include the Openwall, RSX, kNoX, and Exec Shield kernel patches [8, 12, 6, 5]. RSX, kNoX, and Exec Shield can also mark areas of the `malloc()`'d heap as non-executable.

Safe Libraries A safe library re-implements library routines that contain known security vulnerabilities. An application is either statically or dynamically linked to the safe library; every call to the vulnerable routine is then dispatched to the safe version of the routine.

Libsafe [2] is a typical safe library implementation. Libsafe is a dynamically loadable library that intercepts all function calls made to C standard library functions known to be vulnerable to buffer overflow attacks, such as `gets()` and `strcpy()`. A substitute version of each unsafe routine implements the original functionality, but in a manner that ensures any buffer overflows are contained within the current stack frame.

Libsafe does not prevent local variables within the current stack frame from being overwritten. Furthermore, libsafe only protects calls to functions in the standard C library; calls to functions in other libraries are not protected.

2.2.3 Choosing an Appropriate Stack Protection Mechanism

Deciding which stack protection mechanism to use for a particular application can be difficult. Such a decision must take into account performance constraints, source code availability, and other factors.

When writing a new application, full protection against buffer overflows can be obtained by using a "safe language", such as Java or Scheme. If you must use an unsafe language, such as C or C++, consider using a bounds-checking compiler or annotation-assisted static analysis tool.

When protecting an existing piece of software written in C or C++, try recompiling the program using a bounds-checking compiler. If the performance of the recompiled application is insufficient, consider recompiling the application using a canary-based protection tool, such as SSP or SSP+ (the tool presented in this paper).

If you do not have the source code of the program you are trying to protect, link the program to a dynamically loadable library, such as libsafe or libverify.

Of all of the stack protection mechanisms presented in this section, only safe languages, bounds-checking compilers, and annotation-assisted static analysis offer complete protection against buffer overflows. If you cannot use one of these three solutions, use a layered approach to buffer overflow protection for increased security. For example, you could recompile your program using SSP or SSP+, link the program to libsafe, and use an operating system with a non-executable stack; each of these mechanisms protects against a different set of attacks.

References

[1] Aleph One. Smashing the stack for fun and profit. *Phrack Magazine 7*, 49 (1996), File 14. Appeared also on bugtraq (`http://www.securityfocus.com/templates/archive.pike?list=1&;date=1996-%11-08&msg=Pine.LNX.3.91.961109134601.15637b-100000@underground.org`).

[2] Baratloo, A., Singh, N., and Tsai, T. Transparent run-time defense against stack-smashing attacks. In *Proceedings of the 2000 USENIX Annual Technical Conference (USENIX-00)* (Berkeley, CA, June 18–23 2000), USENIX Ass., pp. 251–262.

[3] Cowan, C., Wagle, P., Pu, C., Beattie, S., and Walpole, J. Buffer overflows: Attacks and defenses for the vulnerability of the decade. In *Proceedings of the DARPA Information Survivability Conference and Exposition (DISCEX 2000)* (Jan. 2000), IEEE Computer Society Press.

[4] Etoh, H. Gcc extension for protecting applications from stack-smashing attacks. Published online at `http://www.trl.ibm.com/projects/security/ssp/`. Link last checked on May 6, 2003.

[5] Ingo. "exec shield", new linux security feature. Published online at `http://people.redhat.com/mingo/exec-shield/ANNOUNCE-exec-shield`. Link last checked on May 7, 2003.

[6] ISEC Security Research. knox – implementation of non-executable page protection mechanism. Published online at `http://isec.pl/projects/knox/knox.html`. Link last checked on May 7, 2003.

[7] Larochelle, D., and Evans, D. Statically detecting likely buffer overflow vulnerabilities. In *Proceedings of the Tenth USENIX Security Symposium, August 13–17, 2001, Washington, DC, USA* (Berkeley, CA, USA, 2001), USENIX, Ed., USENIX.

[8] Openwall Project. Linux kernel patch from the openwall project. Published online at `http://www.openwall.com/linux/`. Link last checked on May 7, 2003.

[9] Ramarao, P., Shi, Y., and Park, Y.-J. Detection of memory access errors using bounds checking. Tech. Rep. CPRE 586, Iowa State University, Date unknown.

[10] Richarte, G. Four different tricks to bypass stackshield and stackguard protection. Tech. rep., Core Security Technologies, Apr. 2002.

[11] Solar Designer. Getting around non-executable stack (and fix). Published online at `http://spisa.act.uji.es/spi/progs/codigo/www.hack.co.za/papers/advanced%overflows/ret-libc.txt`. Link last checked on May 7, 2003.

[12] Starzetz, P. Rsx. Published online at `http://www.starzetz.com/software/rsx/`. Link last checked on May 7, 2003.

[13] suk Lhee, K., and Chapin, S. J. Type-assisted dynamic buffer overflow detection. In *Proceedings of the 11th USENIX Security Symposium 2002, August 5–9, 2002, San Francisco, CA* (Berkeley, CA, USA, 2002), USENIX, Ed., USENIX, pp. 81–89.

[14] WireX Communications, Inc. Wirex announces the commercial release of immunix secured linux 7+. Published online at `http://immunix.org/`. Link last checked on May 6, 2003.

A Lab Summary Page

Target audience: Students in CMSC 345.

Length: Up to 1 lecture, $2 - 2\frac{3}{4}$ hours lab time.
Part I — All Your Stack Are Belong to Us: $45 - 60$ minutes
Part II — A Miner's Best Friend: $35 - 45$ minutes
Part III — An Easy 'A': $45 - 60$ minutes

Subject: Buffer overflow defense.

Student prerequisites:

- Familiarity with the C programming language.
- Familiarity with the Linux command line terminal and a Linux text editor.
- Familiarity with the basic concept of a buffer overflow attack (from lecture).
- Familiarity with the basic idea of canary-based stack protection mechanisms (from lecture).

Student learning objectives:

- Students will be able to describe how a stack smashing buffer overflow attack works.
- Students will be able to protect a C program against stack smashing attacks by using SSP, a modified version of GCC that inserts canary values onto the runtime stack.
- Students will be able to identify the limitations of canary–based buffer overflow protection mechanisms.

Student deliverables:

- Writeup: about 2–3 pages, guided by questions at the end of the lab.

Instructor work required:

- Present a lecture on buffer overflow attacks and defense before the lab.
- Setup lab.
- Collect and grade writeups.

B Student Instructions

B.1 Part I — All Your Stack Are Belong to Us

B.1.1 Background

You have received word that Dr. Evil has hatched another dastardly plan to take over the UMBC campus. Dr. Evil is planning to break into UMBC's new computer-controlled safe. Once she steals UMBC's funds, tuition will skyrocket, bringing UMBC to its knees.

UMBC's chief of security has asked you to examine the C program that controls the safe.

B.1.2 Instructions

1. Startup your computer in Red Hat Linux.
2. Login to your computer using the *cdl* account and password supplied to you.
3. Open a terminal window and `cd` to the directory ∼`/buffer/withdraw`.
4. Using your favorite text editor (cough, *emacs*, cough), open the file `withdraw.c`.
5. Examine the source code for the `withdraw` program.

   Question 1 *Describe the* `withdraw` *program. What exactly does the program do?*

6. Compile the `withdraw` program *without* the Stack Smashing Protector:

   `gcc -o withdraw withdraw.c`

   (which is equivalent to '`gcc -fno-stack-protector -o withdraw withdraw.c`')
7. `gcc -o open open.c`
8. `./open | ./withdraw`

B.2 Part II — A Miner's Best Friend

B.2.1 Background

B.2.2 Instructions

1. Recompile the `withdraw` program using the Stack Smashing Protector:

   `gcc -fstack-protector -o withdraw withdraw.c`
2. Before you run the SSP–protected version of `withdraw`, answer the following question:

   Question 2 *What behavior do you expect from the protected version of the* `withdraw` *program? Justify your answer.*

3. Run the `withdraw` program again.
4. `gcc -o open_ssp open_ssp.c`
5. `./open_ssp | ./withdraw`

   Question 3 *How did the protected version of the* `withdraw` *program behave? Were you able to successfully predict this behavior in the previous question? If not, explain the discrepancy.*

B.3 Part III — An Easy 'A'

B.3.1 Background

B.3.2 Instructions

1. In your terminal window, `cd` to the directory `~/buffer/grader`.
2. Open the file `easy_grader.c` in your favorite text editor.
3. Examine the source code for the `easy_grader` program.

   Question 4 *Describe the* ***`easy_grader`*** *program. What exactly does the program do?*

4. In a minute you are going to run the `easy_grader` program, using a long string composed entirely of the character 'A' as the program's input (*i.e.*, '`AAAAAAAAA...AAAAAA`').

   Question 5 *What will be the behavior of the* ***`easy_grader`*** *program with this input string? Justify your answer.*

5. Compile the `easy_grader` program *without* the Stack Smashing Protector:

   ```
   gcc -o easy_grader easy_grader.c
   ```

   For this exercise you should ignore the GCC warning related to the use of the unsafe function `gets()`[1].

6. Run the `easy_grader` program from the terminal:

   ```
   ./easy_grader
   ```

7. `easy_grader` will output some text and then wait for your input. Type a long string of uppercase 'A's (at least 90 characters), then press the *Enter* key:

   ```
   AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
   ```

   Question 6 *How did the* ***`easy_grader`*** *program behave with this input string? Did the program's behavior match your predictions in the previous question? If not, explain the discrepancy.*

8. Recompile the `easy_grader` program, still *without* the Stack Smashing Protector, but this time with code optimizations turned on:

   ```
   gcc -O2 -o easy_grader easy_grader.c
   ```

   Make sure you use the flag '-O2' (Oh-two), *not* '-02' (zero-two).

   Question 7 *How will the code optimizations affect the behavior of the* ***`easy_grader`*** *program, when passed a long string of 'A's as input?*

   Hint: The optimized code saves pointers passed as function arguments in CPU registers instead of on the stack.

9. Run the `easy_grader` program once again, using a long string of 'A's as input.

   Question 8 *How did the optimized version of the* ***`easy_grader`*** *program behave? Were you able to successfully predict this behavior in the previous question? If not, explain the discrepancy.*

10. Still using the -O2 optimization flag, recompile the `easy_grader` program using the Stack Smashing Protector:

    ```
    gcc -fstack-protector -O2 -o easy_grader easy_grader.c
    ```

[1] [explain why gets() is unsafe]

11. Before you run the SSP–protected version of `easy_grader` with the same input string, answer the following question:

    Question 9 *What behavior do you expect from the protected version of the* ***`easy_grader`*** *program? Justify your answer.*

12. Run the SSP–protected version of `easy_grader` from the terminal. Once again provide a long string of capital 'A's as input.

    Question 10 *How did the protected version of the* ***`easy_grader`*** *program behave? Were you able to successfully predict this behavior in the previous question? If not, explain the discrepancy.*

    Hint: When are canary values checked in SSP–protected programs?

    Question 11 *What does the behavior of the* ***`easy_grader`*** *program tell you about the security provided by canary-based stack protection mechanisms?*

C Source Code

C.1 Parts I and II — All Your Stack/Miner's Best Friend

C.1.1 withdraw.c

```
/* withdraw.c */

#define DEBUG
#define PASSWD "openup!"


void open_safe() {
  printf("Opening safe door...\n");

  /* code to open safe goes here */

  printf("Safe opened successfully\n");
}

void verify_passwd() {
  char pass_buf[10];
  int i;

  for (i = 0; i < 10; i++) {
    pass_buf[i] = 0;
  }


#ifdef DEBUG
  printf("** Stack dump before verifying password **\n");
  printf("  Address        Value \n");

  for (i = 20; i >= 0; i--) {
    printf("0x%x => 0x%8.8x\n", &(((long *) pass_buf)[i]), ((long *) pass_buf)[i]);
  }

  printf("\n\n");
#endif

  printf("Please enter your password:\n");

  gets(pass_buf);

  if (strcmp(pass_buf, PASSWD) == 0) {
    open_safe();
  } else {
    printf("Bad password!\n");
  }
```

```
#ifdef DEBUG
  printf("\n\n");
  printf("** Stack dump after verifying password **\n");
  printf("  Address        Value \n");

  for (i = 20; i >= 0; i--) {
    printf("0x%x => 0x%8.8x\n", &(((long *) pass_buf)[i]), ((long *) pass_buf)[i]);
  }

  printf("\n\n");

  printf("&open_safe = 0x%8.8x\n", &open_safe);
#endif
}


int main() {
  verify_passwd();
}
```

C.1.2 open.c

```
/* open.c */

int main() {
  int i;

  char addr[] = "\x98\x83\x04\x08";

  for (i = 0; i<8; i++) {
    printf(addr);
  }
}
```

C.1.3 open_ssp.c

```
/* open_ssp.c */

int main() {
  int i;

  char addr[] = "\x88\x87\x04\x08";

  for (i = 0; i<8; i++) {
    printf(addr);
  }
}
```

C.2 Part III — An Easy 'A'

C.2.1 easy_grader.c

```
/* easy_grader.c */

#define DEBUG
#define NUM_STUDENTS 3

typedef struct {
  char grade;
  char name[10];
} grade_struct;

void print_grade(grade_struct* grades) {
  char name[10];
  int i;

  printf("Welcome to UMBC Easy Grader 1.0\n\n");
  printf("Please enter your last name:\n");

  gets(name);

#ifdef DEBUG
  printf("** Stack dump after getting student name **\n");
  printf("  Address        Value \n");

  for (i = 20; i >= 0; i--) {
    printf("0x%x => 0x%8.8x\n", &(((long *) grades)[i]), ((long *) grades)[i]);
  }
#endif

  for (i = 0; i < NUM_STUDENTS; i++) {
    if (strcmp(name, grades[i].name) == 0) {
      printf("Printing official CMSC 345 course grade for student %s...\n", name);
      printf("Student %s earned an '%c' in CMSC 345.\n", name, grades[i].grade);
      return;
    }
  }

  printf("Could not find a grade for student %s\n", name);

  printf("Please enter your student number:\n");
  i = getchar();
  i -= 0x30;

  printf("Printing official CMSC 345 course grade for student %s...\n", grades[i].name);
  printf("Student %s earned an '%c' in CMSC 345.\n", grades[i].name, grades[i].grade);
}
```

```
int main() {

  grade_struct course_grades[NUM_STUDENTS];
  int i;

  strcpy(course_grades[0].name, "evil");
  strcpy(course_grades[1].name, "smith");
  strcpy(course_grades[2].name, "jones");

  course_grades[0].grade = 'D';
  course_grades[1].grade = 'B';
  course_grades[2].grade = 'C';

#ifdef DEBUG
  printf("** Stack dump before calling print_grade **\n");
  printf("  Address        Value \n");

  for (i = 20; i >= 0; i--) {
    printf("0x%x => 0x%8.8x\n", &(((long *) course_grades)[i]), ((long *) course_grades)[i]);
  }
#endif

  print_grade(course_grades);
}
```

D Instructor/TA Instructions

D.1 Setup

Background: Students should be introduced to the basic ideas of buffer overflow attacks and canary-based defenses against these attacks through a classroom lecture.

Lab Equipment: Standard CDL image (Red Hat Linux 9 + SSP–modified version of GCC) on laptops. Network connectivity is not required.

Handouts: Student instructions.

Split students into pairs; this is not strictly necessary, but may improve discussion and analysis.

D.2 Running the Lab

D.2.1 Part I — All Your Stack Are Belong to Us

D.2.2 Part II — A Miner's Best Friend

D.2.3 Part III — An Easy 'A'

D.3 Exception Handling

- Double check the use of optimization flags. optimization flags change the behavior of code protected with SSP!
- Make sure students use the SSP patched version of GCC on their machines ('which gcc' should return '/usr/pp/bin/gcc').
- Beware of upgrading to a new version of GCC/SSP. The behavior of the example programs may change when built with a different version of the compiler.
- Also beware of changing operating systems or CPU architectures. I have noticed different behavior on different architectures.
- In Part III, make sure that the students are using strings with lengths of 90 characters or more for input to the easy_grader program. Short strings will cause the buffer overflow attack to fail.
- Make sure that students compile the programs with exactly the specified GCC command-line parameters. In particular, incorrect use of the optimization flags can change the behavior of programs.
- For Part III, make sure students use the '-O2' (Oh-two) optimization flag, not '-02' (zero-two).

Contents

1 Introduction **2**

2 Background and Previous Work **2**
2.1 Anatomy of a Buffer Overflow Attack . . . 2
2.2 Existing Stack Protection Mechanisms . . . 3
2.2.1 Canary-based Stack Protection Mechanisms . . . 4
2.2.2 Non-canary-based Stack Protection Mechanisms . . . 5
2.2.3 Choosing an Appropriate Stack Protection Mechanism . . . 6

A Lab Summary Page **9**

B Student Instructions **10**
B.1 Part I — All Your Stack Are Belong to Us . . . 10
B.1.1 Background . . . 10
B.1.2 Instructions . . . 10
B.2 Part II — A Miner's Best Friend . . . 10
B.2.1 Background . . . 10
B.2.2 Instructions . . . 10
B.3 Part III — An Easy 'A' . . . 11
B.3.1 Background . . . 11
B.3.2 Instructions . . . 11

C Source Code **13**
C.1 Parts I and II — All Your Stack/Miner's Best Friend . . . 13
C.1.1 `withdraw.c` . . . 13
C.1.2 `open.c` . . . 14
C.1.3 `open_ssp.c` . . . 14
C.2 Part III — An Easy 'A' . . . 15
C.2.1 `easy_grader.c` . . . 15

D Instructor/TA Instructions **17**
D.1 Setup . . . 17
D.2 Running the Lab . . . 17
D.2.1 Part I — All Your Stack Are Belong to Us . . . 17
D.2.2 Part II — A Miner's Best Friend . . . 17
D.2.3 Part III — An Easy 'A' . . . 17
D.3 Exception Handling . . . 17